

Andrew Hall
C S 474
Professor Ventura
April 21st 2024

Predicting Stock Returns With Convolutional Neural Networks

Introduction

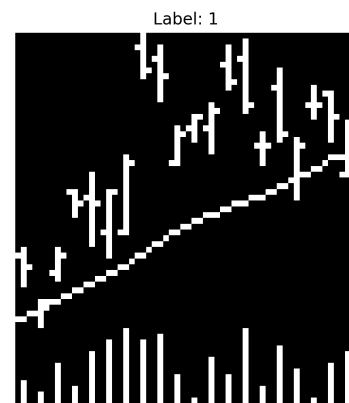
Investors have been using historical price and volume data to predict future stock returns since as early as the 1920s. Methodologies have evolved from rule based timing strategies, to econometric style regression forecasts, and in recent times to machine learning based approaches. Deep learning has been frowned upon by economists for this task due to the lack of interpretability and tendency to overfit. Despite this resistance, some academics have still explored different applications of neural networks in forecasting stock returns. For this project I have replicated the results of “(Re-)Imag(in)ing Price Trends” by Brian Kelly in order to validate the efficacy of Convolutional Neural Networks in active portfolio management.

Data

I used the Center for Research in Security Prices (CRSP) daily security file since it is the gold standard in asset pricing research. This dataset includes open, high, low, and close prices (OHLC) as well as trading volume by shares since 1993. I split the dataset into in-sample (1993-2000) and out of sample (2001-2019) datasets and then further selected the first 80% of the in-sample dataset as the training set and the last 20% as the validation set. No shuffling was done on any of the datasets.

The pricing datasets were then transformed into an image dataset with an image for each date and security pair. Each image contains 20 days of data where each day is represented by 3 pixels. The first pixel contains the open price,

the second contains the high low range, and the third contains the closing price. A moving average price line, and volume bars were also added to the image. The image is scaled to an arbitrary height (64 pixels in our case) such that the volume bars encompass the bottom 20% of the image with the top pixel of the range representing the max volume over the lookback window. The top 80% of the image contains the OHLC bars and the moving average price with the top pixel of the range representing the max price over the lookback window. The label for the image is a binary variable that is equal to 1 if the next 20 day return is positive and 0 otherwise.



Model Architecture

I used a Convolutional Neural Network with three sequential blocks. Each block contained a 2d-convolutional layer, a batch norm layer, a max pooling layer, and a leaky ReLu layer. The 2d-convolutional layers have a kernel size of 5x3 and the max pooling layers have a kernel size of 2x1. This allows for the model to capture relationships along the vertical axis with a width of a single day. After the third block I included a

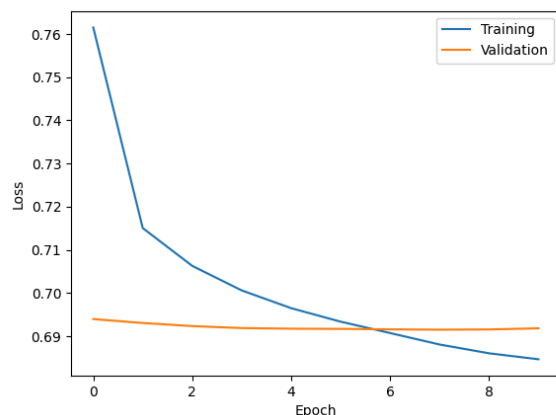
dropout layer and a linear layer with 46,080 input features and a single output feature.

Training

The model was trained using the PyTorch binary cross entropy loss with logits loss function for numerical stability. The model was trained with early stopping contingent on the validation set. If the validation loss didn't improve for 2 consecutive epochs the model terminated training. A batch size of 128 and a learning rate of $1e-5$ were used.

Results

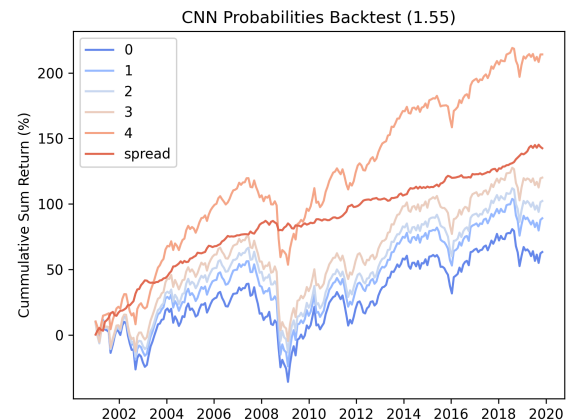
Our model stopped training after 9 epochs due to the early stopping constraint. As expected the validation loss didn't significantly improve as a result of the training and validation sets being fundamentally different. This is a unique aspect of financial markets often characterized by the “efficient market hypothesis”. Essentially if stock returns can be predicted by some feature in the training set, then the market will price this in for the validation set.



The model scored 53.55% accuracy on the test set. However, with the test set having 54.88% positive labels, the model's ability to predict binary returns is in fact below the baseline.

In hopes to better evaluate the efficacy of the model, I took the predicted logits for the test set, passed them through a sigmoid function to create the predicted probabilities of next period

positive returns. I then created 5 portfolios for each day in my testing period where portfolio 5 contained the top quintile of stocks according to their probabilities, and portfolio 1 had the bottom quintile of stocks. I then created a spread portfolio which was long portfolio 10 and short portfolio 1.



To measure the performance of the spread portfolio I computed the sharpe ratio, which is the average daily return divided by the standard deviation of daily returns. The CNN achieved a sharpe ratio of 1.55 on the test set, which far outperforms the market average of .4-.5. These results have proved the validity in using deep learning to forecast stock returns.

Conclusion

For this project I have replicated the results of “(Re-)Imag(in)ing Price Trends” by Brian Kelly, showing that deep learning models such as convolutional neural networks are able to effectively forecast stock returns. I did this by converting tabular pricing data into an image dataset that a CNN could then use to learn the relationship between historical price trends and future returns.

Time Log

Date	Hours	Notes
2/4/25	1	Read "From Time Series to Images" paper
2/6/25	1	Read "(Re-)imag(in)ing Price Trends" paper
2/11/25	1.5	Downloaded and cleaned CRSP daily security file dataset from WRDS
2/13/25	1.5	Built a system for gathering technical indicator features
2/25/25	1.5	Experimented with different methods for creating images
2/27/25	1.5	Worked on price trend image generator
3/1/25	2	Finished building image generator
3/4/25	2	Had to rewrite image generator to avoid maxing out storage resources
3/6/25	2	Built a simple CNN as a proof of concept and implemented pytorch dataset
3/11/25	2	Worked on replicating CNN architecture from paper
3/13/25	1.5	Got a working replication of the paper CNN model
3/15/25	1	Created a script for generating a loss chart
3/18/25	1.5	Ran an experiment on learning rate vs. test accuracy
3/20/25	2	Ran an experiment on number of epochs vs. test accuracy
3/22/25	1.5	Added early stopping and random seeds to experiment framework
3/29/25	1.5	Reran earlier experiments with new experiment framework
4/1/25	2	Created a script for backtesting strategy on test set
4/3/25	2	Created script for generating filter .png files
4/8/25	1	Reran earlier experiments with backtest scripts
4/10/25	2	Implemented PyTorch Lightning framework to reduce boilerplate code
4/12/25	2	Ran an experiment on shuffling training set vs. sharpe ratio
Total	34	